

Building Java Programs

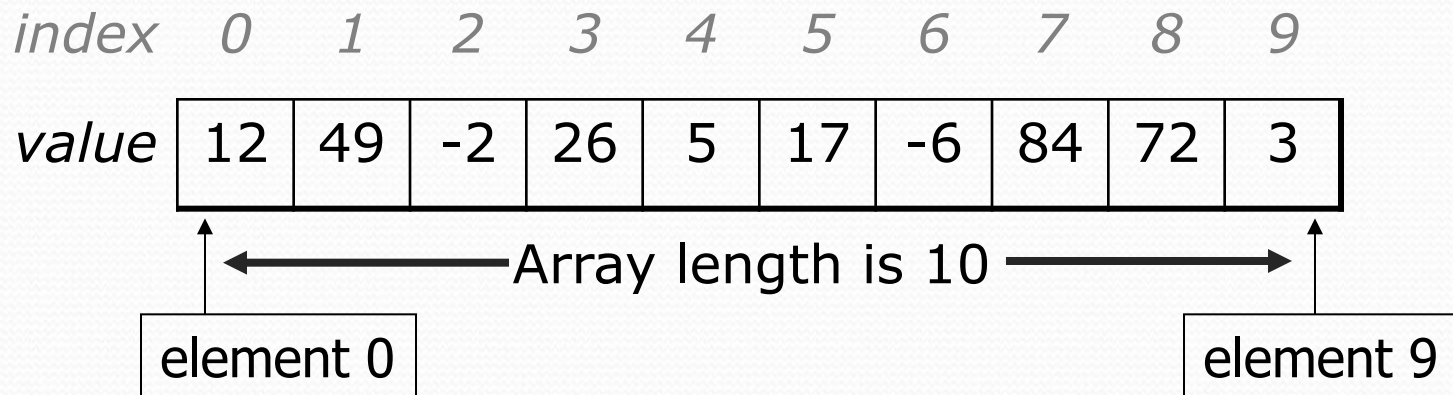
Arrays



Arrays

What is an Array?

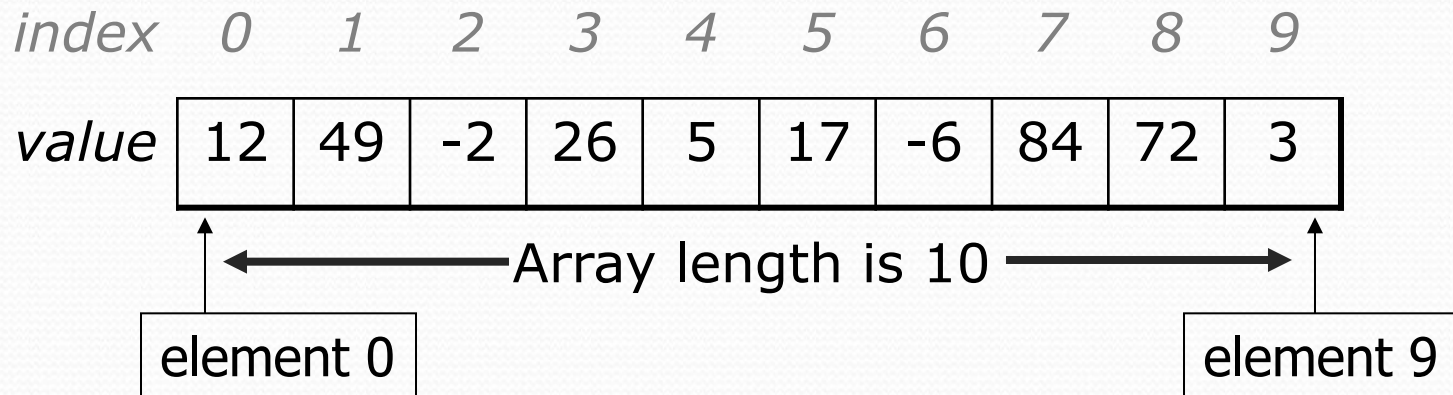
- An array is collection of values of the same data type stored in a container object.
- Can be any number of values.
- Length of the array is set when the array is declared.
- Size is fixed once the array is declared.



Arrays, cont.

Array Elements

- Every value is called an element.
- Each element is accessed by an index.
- Index must be an integer.
- Index always starts at 0.



Declaring an Array

- There are three parts to declaring an array:
 - Data type
 - Variable name
 - Array Size
- An array declaration can be done in one or two lines
- Both examples below are equivalent array declarations.

//declare an array using two lines of codes

```
data_type[] variable_name;  
variable_name = new data_type[ size ];
```

//declare an array using one line of codes

```
data_type[] variable_name = new data_type[ size ];
```

Declaring an Array, cont.

- The length can be any integer expression.

```
int x = 2 * 3 + 1;
```

```
int[] data = new int[x % 5 + 2];
```

Type	Default value
int	0
double	0.0
boolean	false
String or other object	null (means, "no object")

Accessing elements

name[**index**] *// access*
name[**index**] = **value**; *// modify*

- Example:

```
int[] numbers = new int[10];  
numbers[0] = 27;  
numbers[3] = -6;
```

```
System.out.println(numbers[0]); // display 27  
System.out.println(numbers[3]); // display -6
```

<i>index</i>	0	1	2	3	4	5	6	7	8	9
<i>value</i>	27	0	0	-6	0	0	0	0	0	0

Accessing array elements

```
int[] numbers = new int[8];  
numbers[1] = 3; //assigned the value of 3 to index 1  
numbers[4] = 99; //assigned the value of 99 to index 4  
numbers[6] = 2; // assigned the value of 2 to index 6  
  
int x = numbers[1]; //assigned the element value of index 1 to x  
numbers[x] = 42; //assigned 42 to element index 3  
numbers[numbers[6]] = 11; // use numbers[6] as index
```

x

3

		index	0	1	2	3	4	5	6	7	8	9
numbers	value	0	4	11	42	99	0	2	0	0	0	0

Arrays of other types

```
double[] results = new double[5];  
results[2] = 3.4;  
results[4] = -0.5;
```

<i>index</i>	0	1	2	3	4
<i>value</i>	0.0	0.0	3.4	0.0	-0.5

```
boolean[] tests = new boolean[6];  
tests[3] = true;
```

<i>index</i>	0	1	2	3	4	5
<i>value</i>	false	false	false	true	false	false

Out-of-bounds

- Legal indexes: between **0** and the **array's length - 1**.
 - Reading or writing any index outside this range will throw an `ArrayIndexOutOfBoundsException`.

- Example:

```
int[] data = new int[10];  
System.out.println(data[0]);           // okay  
System.out.println(data[9]);           // okay  
System.out.println(data[-1]);          // exception  
System.out.println(data[10]);         // exception
```

<i>index</i>	0	1	2	3	4	5	6	7	8	9
<i>value</i>	0	0	0	0	0	0	0	0	0	0

Arrays and `for` loops

- It is common to use `for` loops to access array elements.

```
int[] numbers = new int[10];
```

```
numbers[0] = 74;
```

```
numbers[3] = 99;
```

```
numbers[5] = 50;
```

```
numbers[7] = 26;
```

```
numbers[9] = 85;
```

```
for (int i = 0; i < 5; i++) {
```

```
    System.out.print(numbers[i] + " ");
```

```
}
```

```
System.out.println(); //output:74 0 0 99 0 50 0 26 0 85
```

index 0 1 2 3 4 5 6 7 8 9

numbers

value

74	0	0	99	0	50	0	26	0	85
-----------	---	---	-----------	---	-----------	---	-----------	---	-----------

Arrays and `for` loops

- Sometimes we assign each element a value in a loop.

```
int[] numbers = new int[10];  
for (int i = 0; i < 10; i++) {  
    numbers[i] = 2 * i;  
}
```

	<i>index</i>	0	1	2	3	4	5	6	7	8	9
<i>numbers</i>	<i>value</i>	0	2	4	6	8	10	12	14	16	18

The length field

- An array's `length` field stores its number of elements.

array_name.length

```
for (int i = 0; i < numbers.length; i++) {  
    System.out.print(numbers[i] + " ");  
}
```

// output: 0 2 4 6 8 10 12 14 16 18

- It does not use parentheses like a String's `.length()`.

Weather question

- Use an array to solve the weather problem:

How many days' temperatures? 7

Day 1's high temp: 45

Day 2's high temp: 44

Day 3's high temp: 39

Day 4's high temp: 48

Day 5's high temp: 37

Day 6's high temp: 46

Day 7's high temp: 53

Average temp = 44.6

4 days were above average.

Weather answer

```
// Reads temperatures from the user, computes average and # days above average.
import java.util.*;

public class Weather {
    public static void main(String[] args) {
        Scanner console = new Scanner(System.in);
        System.out.print("How many days' temperatures? ");
        int days = console.nextInt();

        int[] temperatures = new int[days]; // array to store days' temperatures
        int sum = 0;

        for (int i = 0; i < days; i++) { // read/store each day's temperature
            System.out.print("Day " + (i + 1) + "'s high temp: ");
            temperatures[i] = console.nextInt();
            sum += temperatures[i];
        }
        double average = (double) sum / days;

        int count = 0; // see if each day is above average
        for (int i = 0; i < days; i++) {
            if (temperatures[i] > average) {
                count++;
            }
        }

        // report results
        System.out.printf("Average temp = %.1f\n", average);
        System.out.println(count + " days above average");
    }
}
```

Arrays for counting and tallying



A multi-counter problem

- Problem: Examine a large integer and count the number of occurrences of every digit from 0 through 9.
 - Example: The number 229231007 contains:
two 0s, one 1, three 2s, one 7, and one 9.
- We could declare 10 counter variables for this...

```
int counter0, counter1, counter2, counter3, counter4,  
    counter5, counter6, counter7, counter8, counter9;
```

 - Yuck!

A multi-counter problem

- A better solution is to use an array of size 10.
 - The element at index i will store the counter for digit value i .
 - for integer value 229231007, our array should store:

<i>index</i>	0	1	2	3	4	5	6	7	8	9
<i>value</i>	2	1	3	0	0	0	0	1	0	1

- The index at which a value is stored has meaning.
 - Sometimes it doesn't matter.
 - What about the weather case?